

ChatGPT from scratch

SGD

$$w_{k+1} = w_k - \alpha_k \nabla_w \mathcal{L}(w_k)$$

(Stochastic) Gradient Descent

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$w_k = w_{k-1} - \alpha_k \cdot g_k$$

Momentum

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$w_k = w_{k-1} - \alpha_k \cdot \mu_k$$

RMSProp

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

Adaptive step
for each coordinate
(~second-order methods)



$$w_k = w_{k-1} - \alpha_k \cdot \frac{g_k}{\sqrt{v_k + \epsilon}}$$

Bringing everything together: almost Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\mu_k}{\sqrt{\hat{v}_k + \epsilon}}$$

Bringing everything together: Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}}$$

Bringing everything together: Adam

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}}$$

Adam with L_2 -regularization

$$g_k = \nabla_w \mathcal{L}(w_k) + \lambda w_k$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}}$$

AdamW: Adam with L_2 weight decay

$$g_k = \nabla_w \mathcal{L}(w_k)$$

$$\mu_k = \beta_1 \cdot \mu_{k-1} + (1 - \beta_1) g_k$$

← Accumulates previous directions
Makes GD less erratic

$$v_k = \beta_2 \cdot v_{k-1} + (1 - \beta_2) g_k^2$$

← Adaptive step
for each coordinate
(~second-order methods)

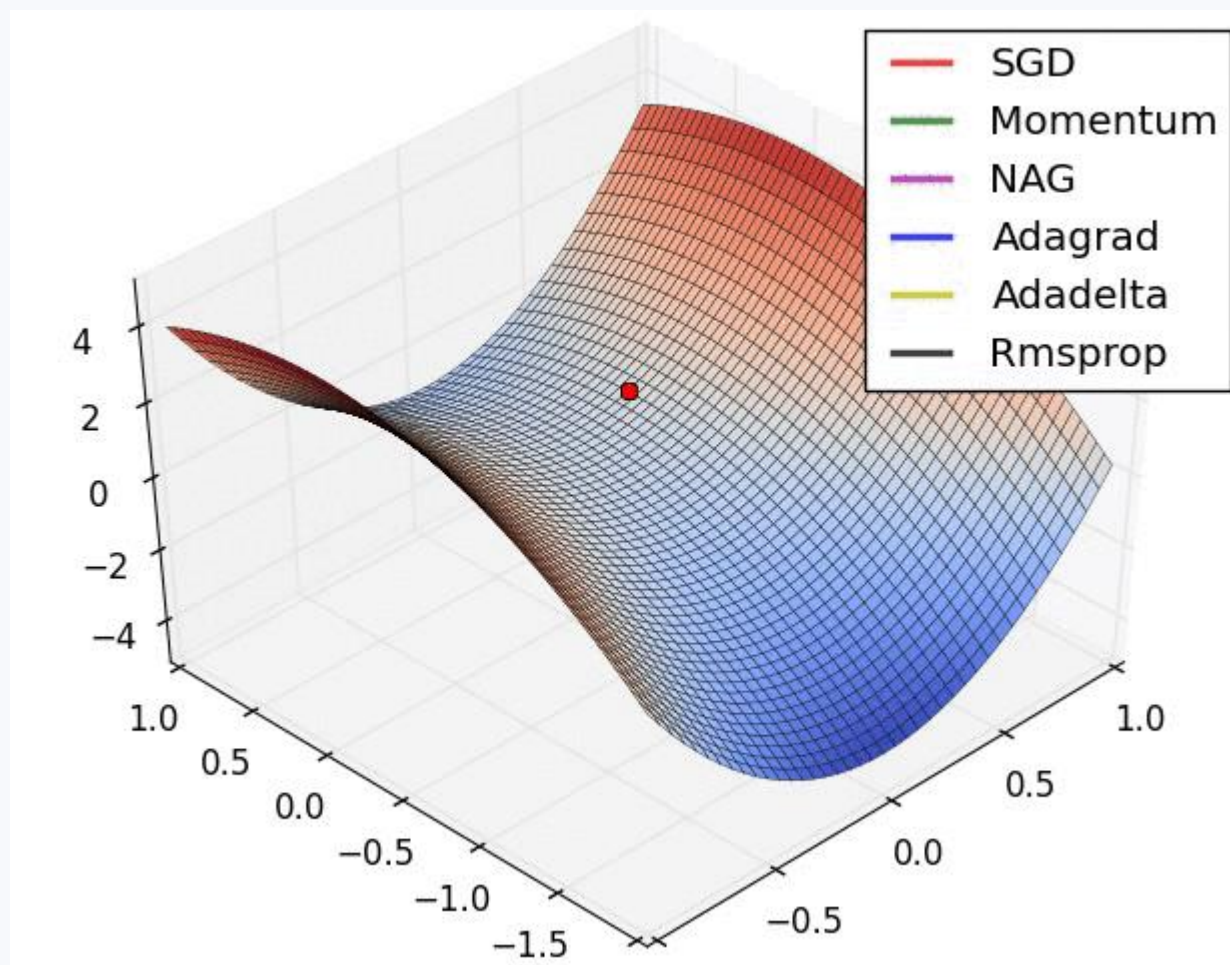
$$\hat{\mu}_k = \frac{\mu_k}{1 - \beta_1^k},$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

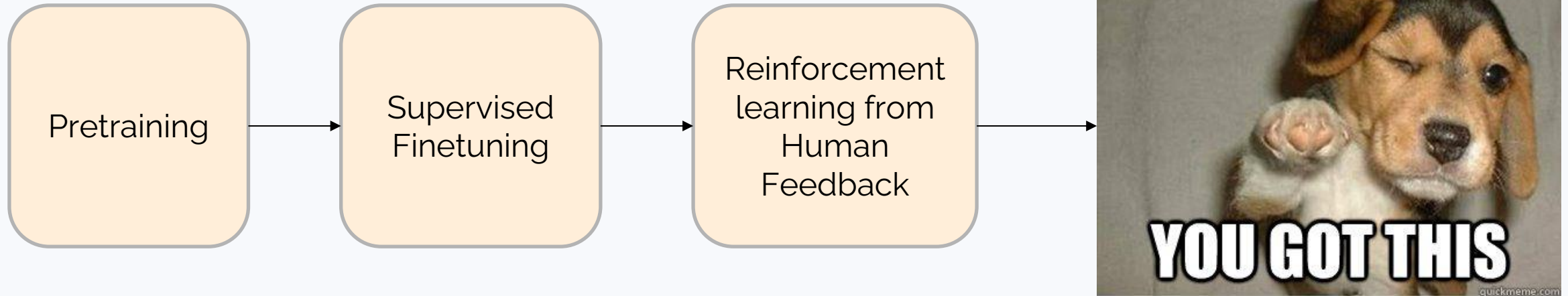
← Bias correction

$$w_k = w_{k-1} - \alpha_k \cdot \left(\frac{\hat{\mu}_k}{\sqrt{\hat{v}_k + \epsilon}} + \lambda w_k \right)$$

Why this all matters

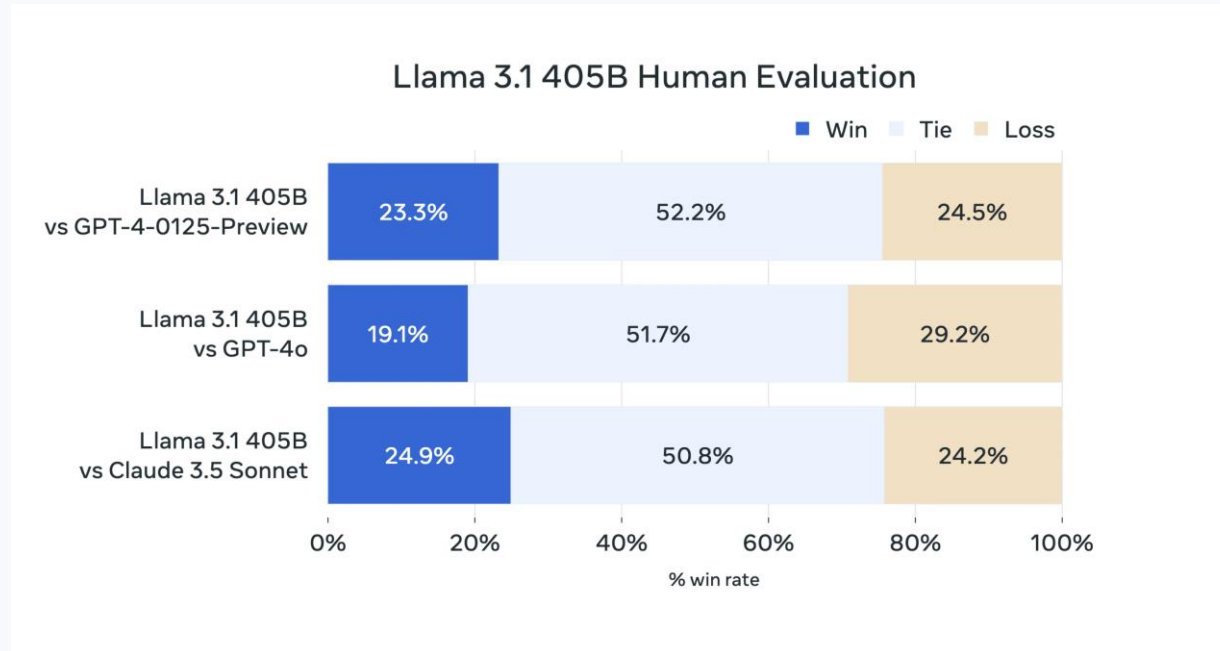


The process



Remark

We will speak about llama pretraining process mostly as we know the exact numbers and can try this model



LLM pretraining

- Model size – billions parameters
- Data – terabytes of data
- Compute – thousands of GPUs



Llama 3.1

Model size: 405 billions of parameters

Data: 120 TB – 15 trillions of tokens

GPUs: up to 16k H100

* The Harry Potter and the Philosopher's Stone contains 109,000 tokens

LLM pretraining from ML perspective

Task: multiclass classification – next token prediction

Loss: cross entropy loss

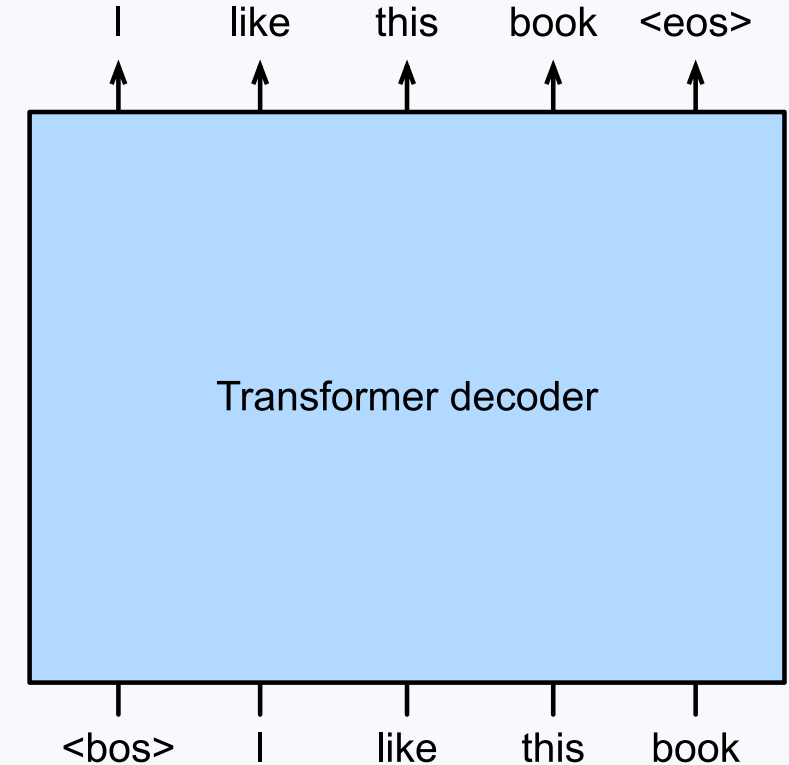
Model architecture: Transformer decoder

LLM = $P(y \mid x)$, x – prompt, y – next token

$$L = \frac{1}{K} \sum_{i=1}^K l_i, \quad l_i = \frac{1}{N} \sum_{i=1}^N I[c_i = \text{next token}] \cdot \log p_i$$

K – number of tokens in example or batch, N – dictionary size

$$K = bs * seq_{len}$$



- Loss is calculated for each token

LLM, how many gpus we need?

- LLM weights: 405 billion parameters
- Optimizer states:
AdamW – 2 states, mu and nu, same size as weights
- Gradients, same size as weights
- Activations, same size as weights
- KV cache for inference

Llama 405b config:

```
"hidden_size": 16384,  
"num_attention_heads": 128,  
"num_hidden_layers": 126,  
"num_key_value_heads": 8,  
"max_position_embeddings": 131072,
```

1 host = 8 GPU, 1 H100 = 80 Gb

LLM, how many gpus we need?

- LLM weights: 405 billion parameters
- Optimizer states:
AdamW – 2 states, mu and nu, same size as weights
- Gradients, same size as weights
- Activations, same size as weights
- KV cache for inference

Llama 405b config:

"hidden_size": 16384,

"num_attention_heads": 128,

"num_hidden_layers": 126,

"num_key_value_heads": 8,

"max_position_embeddings": 131072,

1 host = 8 GPU, 1 H100 = 80 Gb

Training:

- Weights stored in float32
 $405 * 10^9 * 32 / 2^{33} = 1508.74 \text{ Gb}$
- Optimizer states stored in float32
 $2 * 405 * 10^9 * 32 / 2^{33} = 3017.48 \text{ Gb}$
- Gradients in float32
 $405 * 10^9 * 32 / 2^{33} = 1508.74 \text{ Gb}$
- Activations in in bfloat16
 $405 * 10^9 * 16 / 2^{33} = 754.37 \text{ Gb}$

Overall train for bs=1: 6789.33 Gb / 80 Gb = 85 gpus = 11 hosts (likely 16 hosts minimum)

Inference:

- Weights stored in bfloat16
 $405 * 10^9 * 32 / 2^{33} = 754.37 \text{ Gb}$
- KV cache in bfloat16
 $(2 * \text{seq len} = 131072) * (\text{kv_heads} = 8) * (\text{hidden_head_dim} = 128) * (\text{num_layers} = 126) * (\text{bfloat16} = 16) / 2^{33} = 63 \text{ GB}$
- Activations – small constant

Overall inference for bs = 1: (754.37 Gb + 63 Gb) / 80 Gb = 10 gpus = 2 hosts (each 8 gpus)

Context window increasing

Problem:

The Attention time complexity from the input length?

Context window increasing

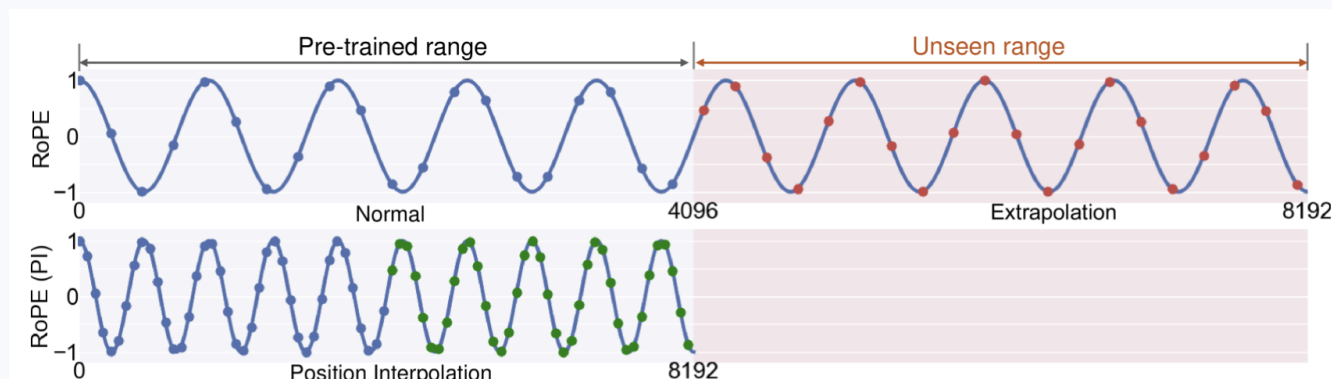
Problem:

The Attention time complexity from the input length is quadratic. So, the calculation complexity will increase k times.

Llama 3.1 context window is 128k tokens.

But the pretraining stage is done on 8k tokens context window.

How? Long-context pretraining



Llama 3.1 405b:

- approximately 800B training tokens used
- 6 stages: 8k -> 16k -> 32k -> 64k -> 128k (some in between)

Pretraining results

Pretrained Llama-3.2-3B generation:

What is the capital of United Kingdom?	(A) London (B) Birmingham (C) Cardiff (D) Edinburgh A. A
prompt	answer

- Why does it look like this?
- What was the main purpose of pretraining?
- What do we want the answer to look like?

Post-training (Alignment)

What for?

To make model work as assistant

To follow instructions not just continue the text

To increase safety

After alignment



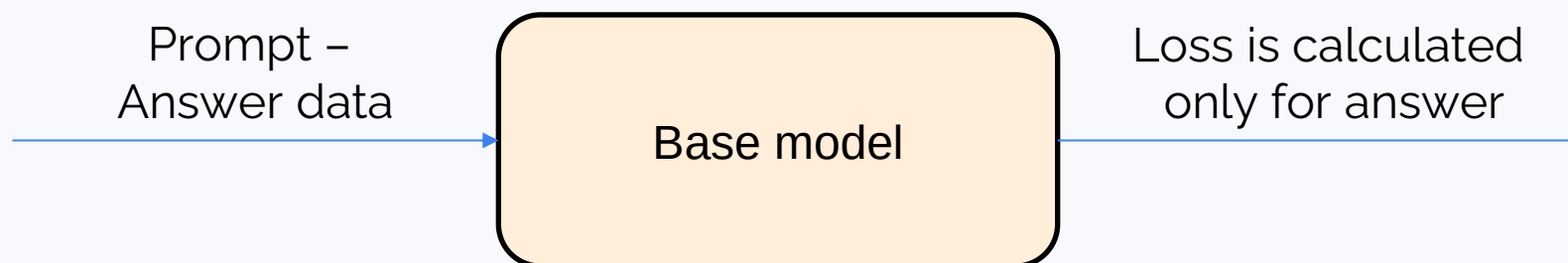
Before alignment



SFT – supervised fine-tuning

SFT = pure data of highest quality.

Format: question answering, assistant format

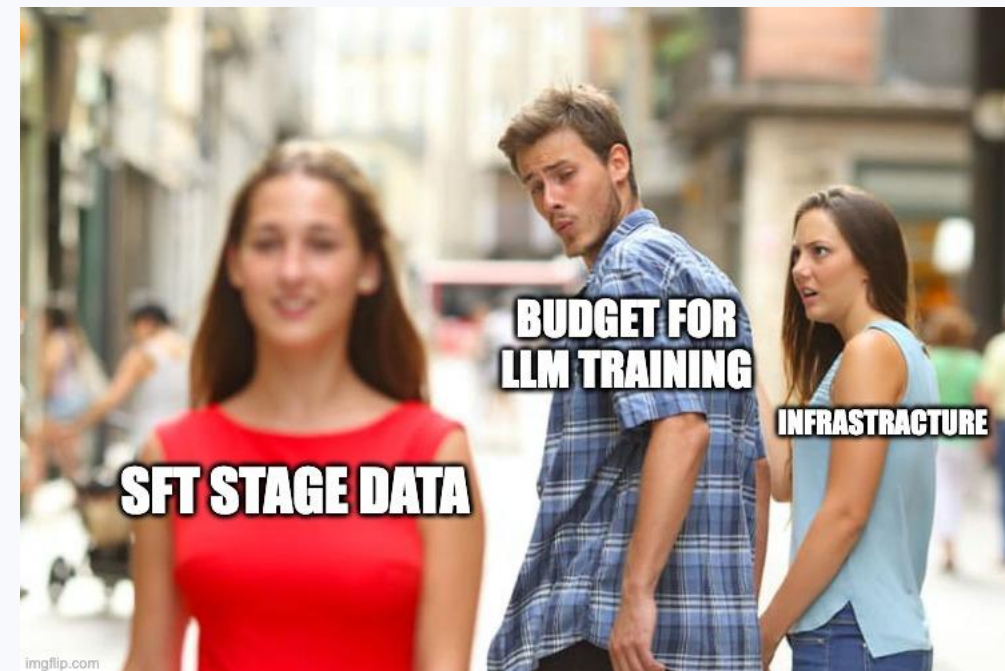


SFT – supervised fine-tuning

What is the problem with SFT?

SFT data is expensive

It's hard for a person to write a good answer



RLHF – reinforcement learning from human feedback

Reinforcement learning setup:

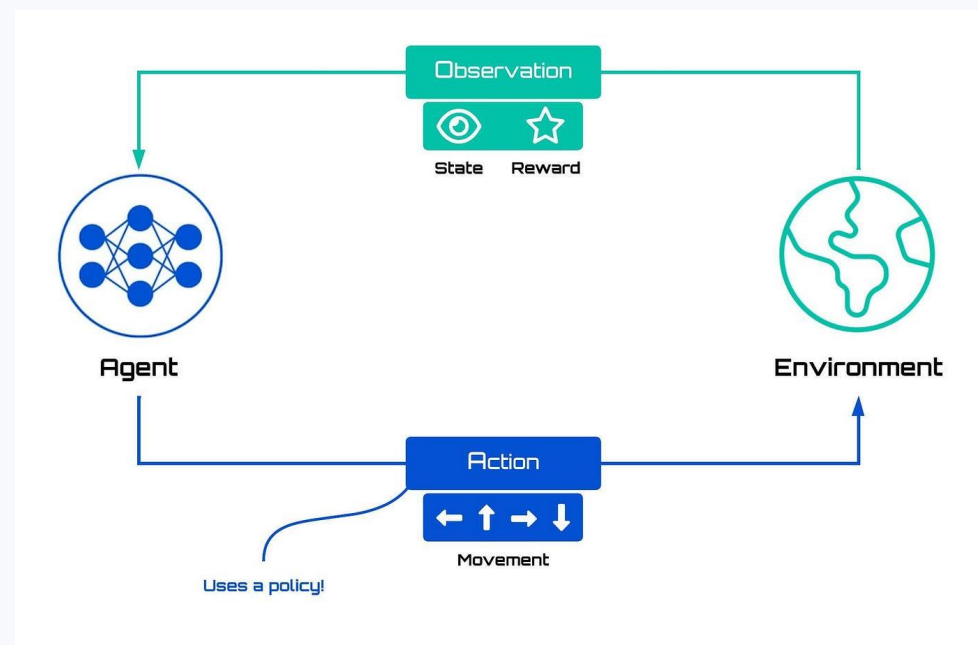
A state represents the current situation of the environment: $s \in S$

$\pi(a|s) = P(a | s)$ policy, mapping states to actions

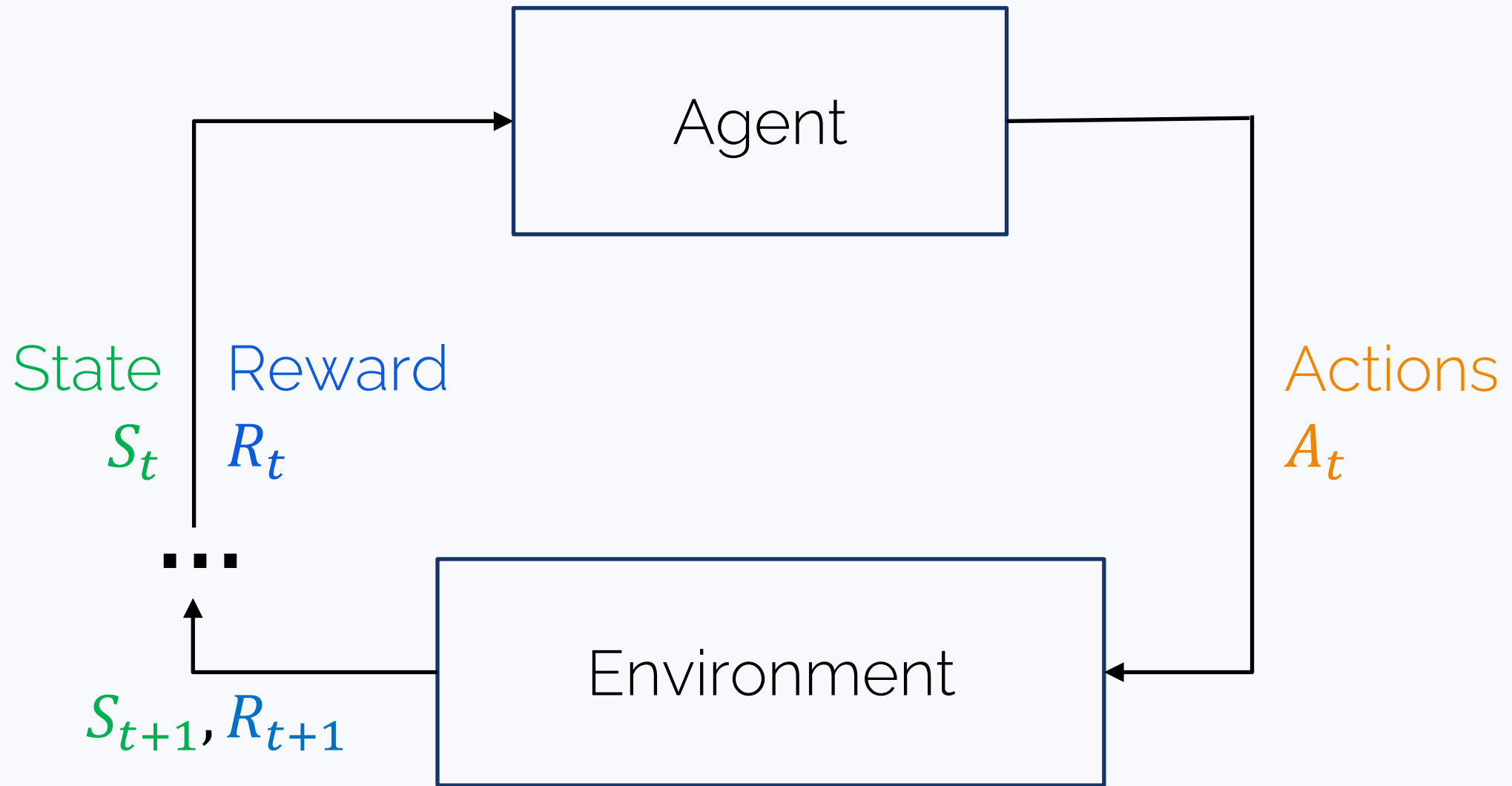
Explanation: Specifies the probability of taking action a when in state s .

The feedback signal received after transitioning to a new state: $R(s, a) \in \mathbb{R}$

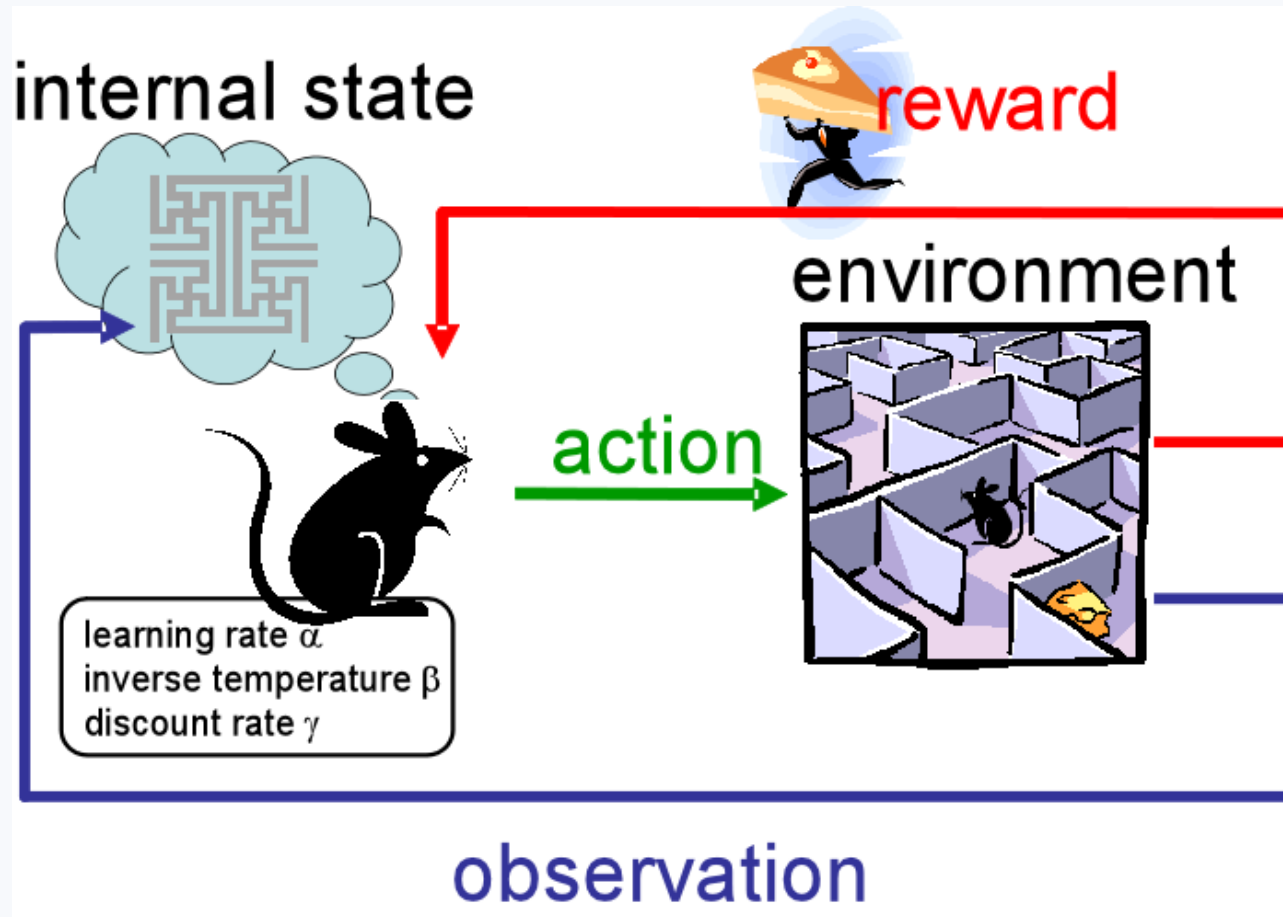
Transition Dynamics ($P(s'|s, a)$): The rules governing how the environment changes state based on the agent's actions



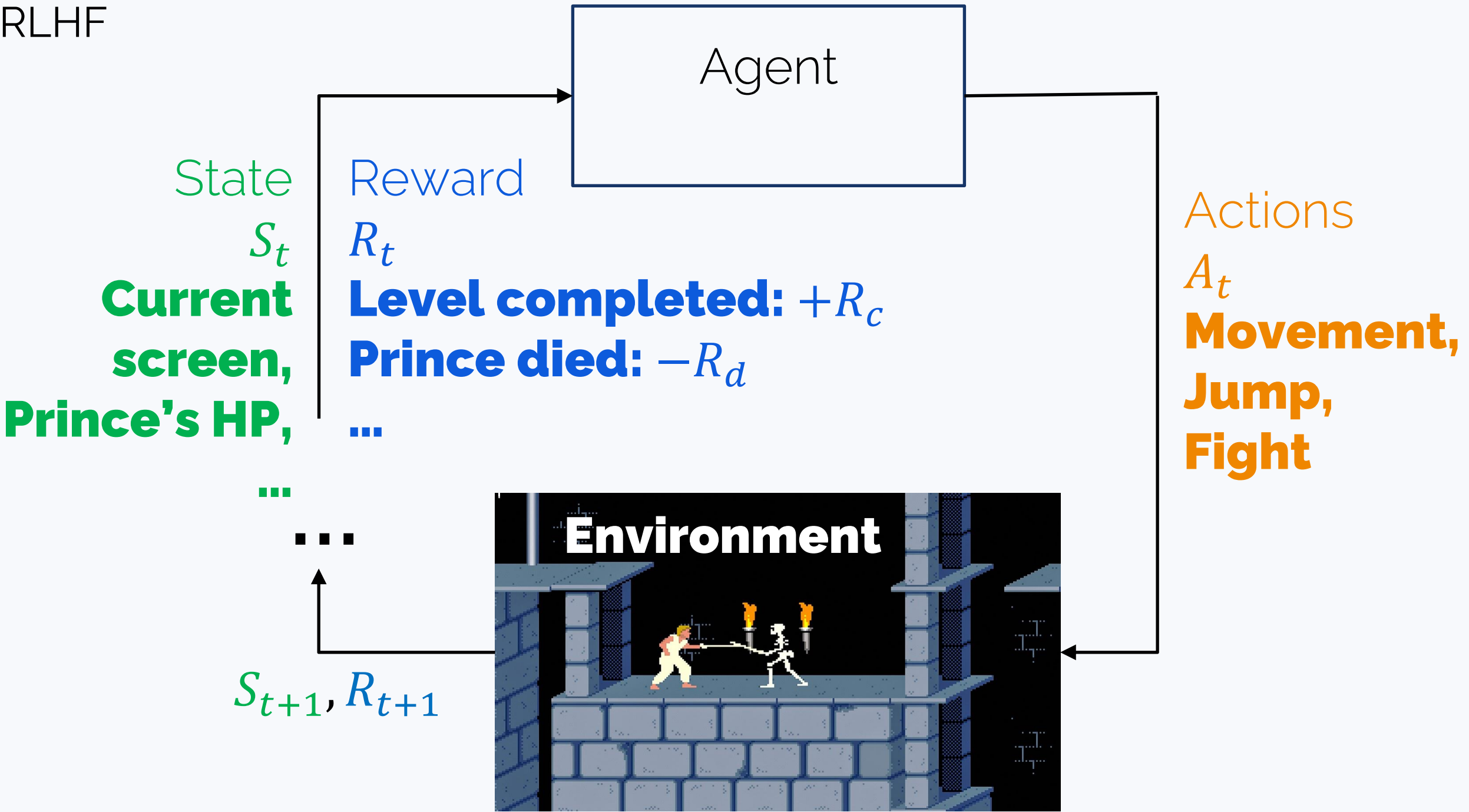
RLHF – reinforcement learning from human feedback



RLHF – reinforcement learning from human feedback



RLHF

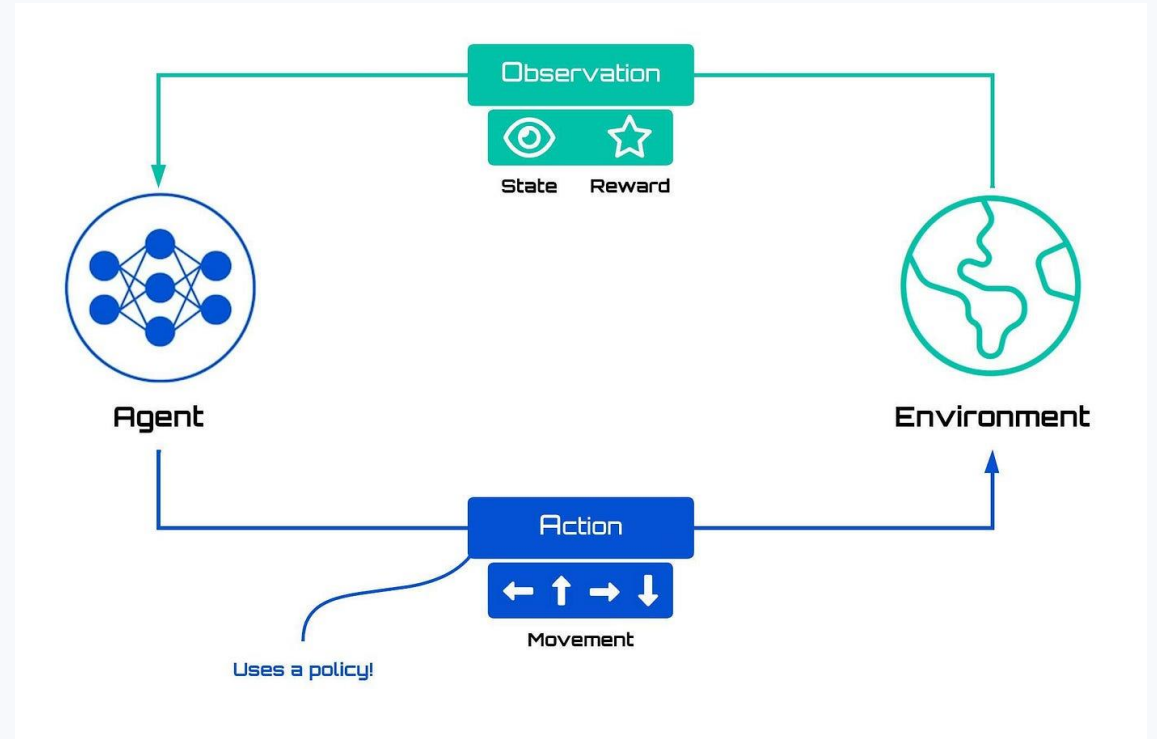


RLHF – reinforcement learning from human feedback

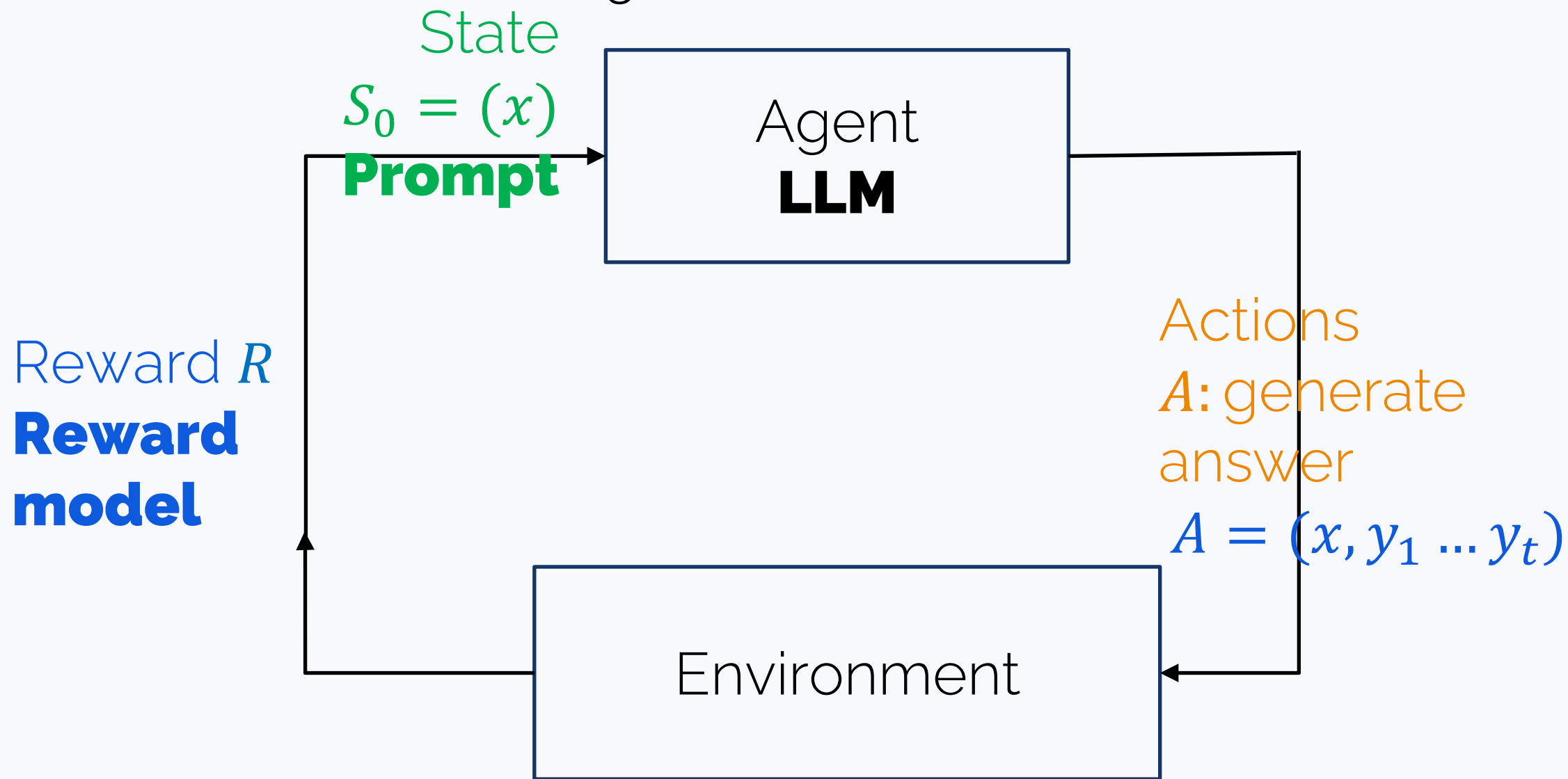
Environment = Reward model

Feedback = Score
(high score = good answer,
low score = bad answer)

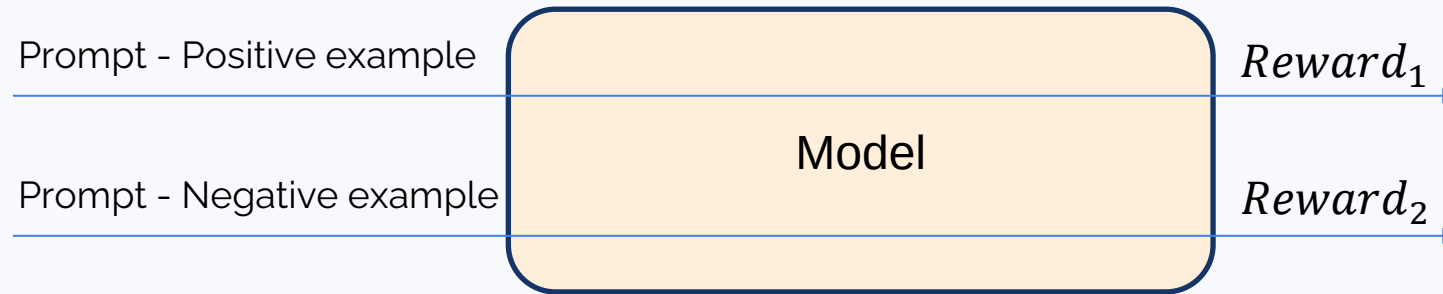
Agent = LLM



RLHF – reinforcement learning from human feedback



Reward model – Bradley-Terry model



What the problem is?

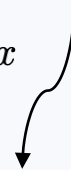
- We can't make absolute reward for every example as we need thousands of data. Moreover, prompts are different
- We have only relative estimations, mostly pairwise data (1 prompt, 2 answers –which one is better)
- For RL we need absolute reward for the answer

Reward model – Bradley-Terry model

We want to train: $r(y \mid x)$

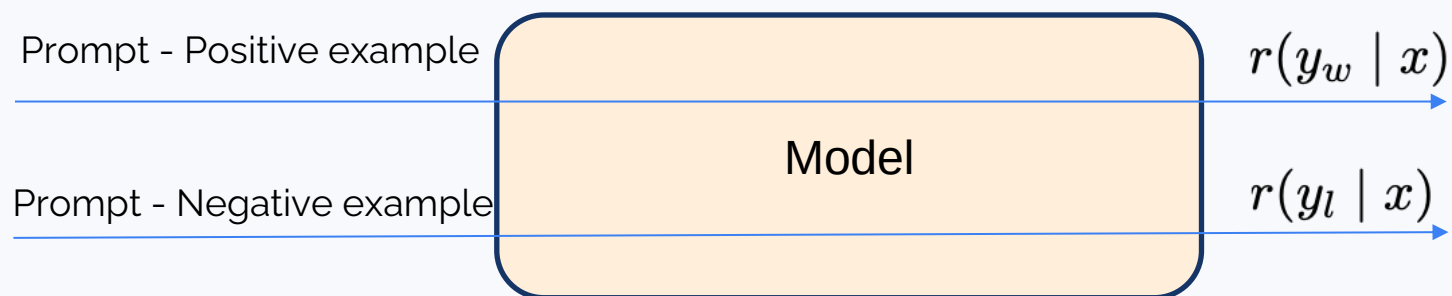
We have pairwise estimation: $x_{\text{preferred}} \succ x_{\text{other}}$

$$P(x_{\text{preferred}} \succ x_{\text{other}}) = \frac{e^{r(x_{\text{preferred}})}}{e^{r(x_{\text{preferred}})} + e^{r(x_{\text{other}})}}$$

$$\begin{array}{l} x_{\text{preferred}} = y_w \mid x \\ x_{\text{other}} = y_l \mid x \end{array}$$


$$\text{Loss} = -\log(\sigma(r(y_w \mid x) - r(y_l \mid x)))$$

Reward model



Requirements:

- Prompt + 2 answers + pairwise estimation

$$\text{Loss} = -\log(\sigma(r(y_w | x) - r(y_l | x)))$$

Loss increases the gap between positive and negative example

RLHF – reinforcement learning from human feedback



RLHF – reinforcement learning from human feedback

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r(x, y)] - \beta \mathcal{D}_{\text{KL}} [\pi_{\theta}(y|x) \parallel \pi_{\text{SFT}}(y|x)]$$

↑
RLHF Objective

↑
Reward
from model

↙
KL divergence

x = prompts

y = LLM generated answers

$\pi_{\theta}(y | x)$ = LLM

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[r(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x)} \right]$$

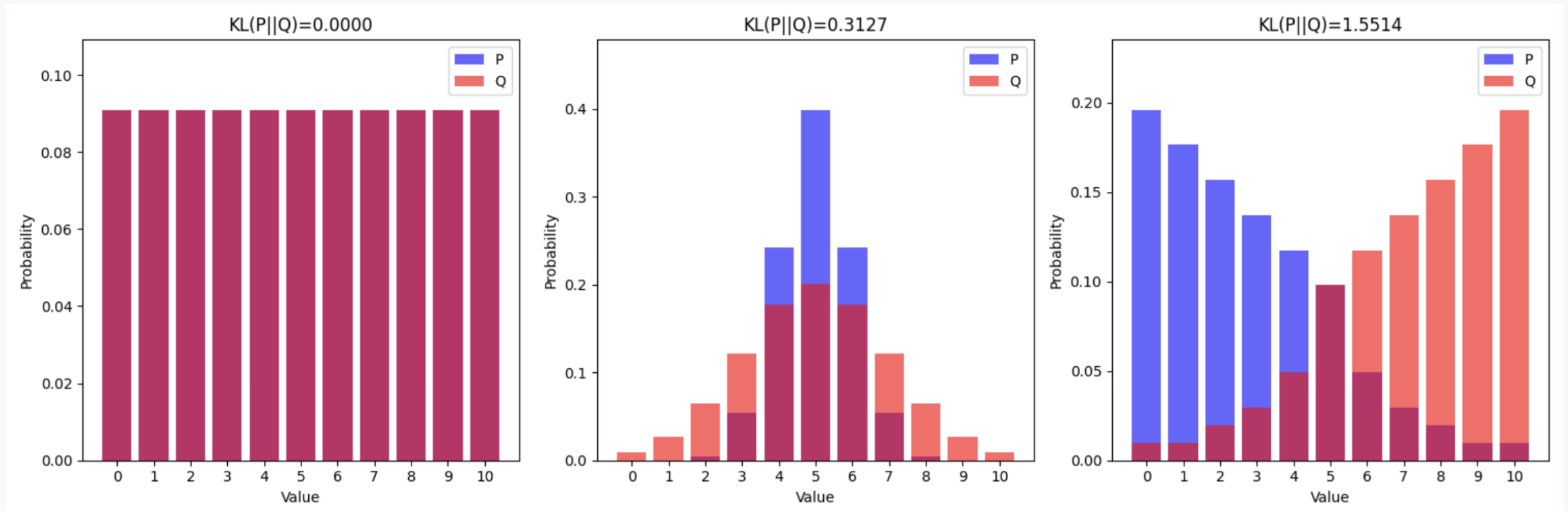
What if there is no KL divergence term?

$$^* \mathcal{D}_{\text{KL}}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i} = \mathbb{E}_p \log \frac{p}{q}$$

KL divergence

KL divergence is a measure of how a probability distribution differs from another probability distribution

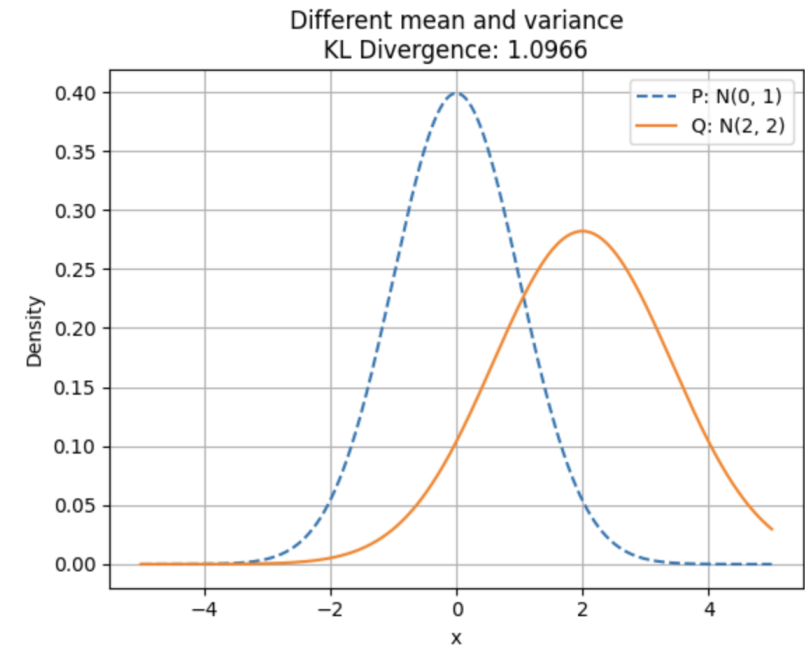
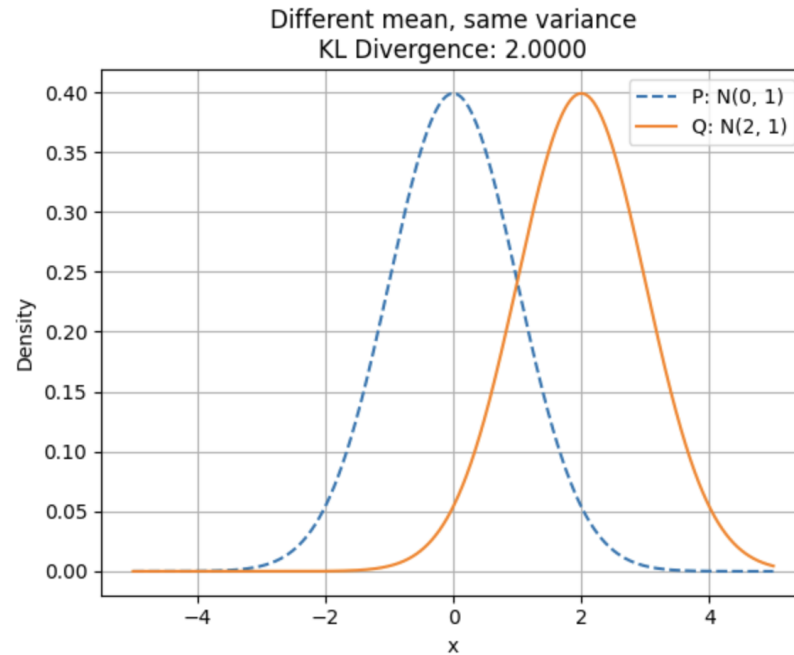
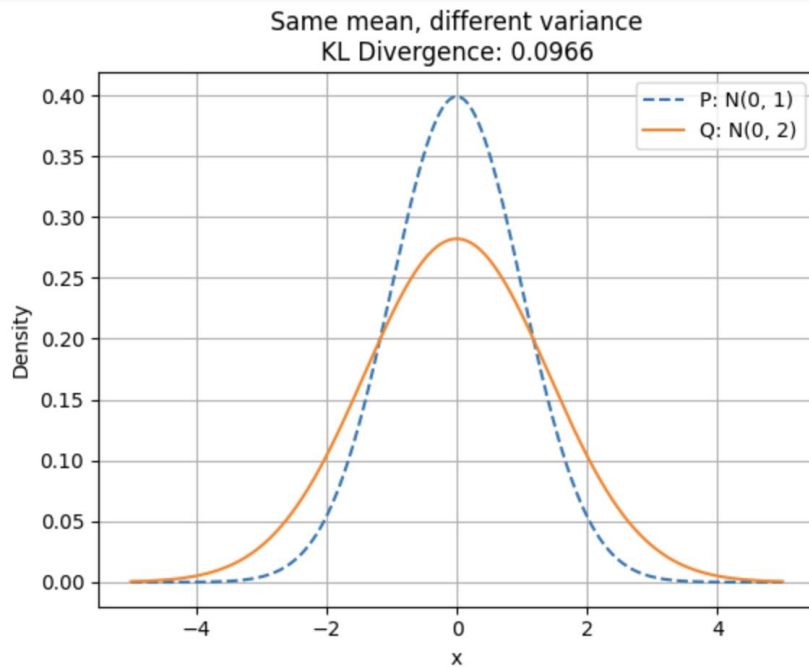
$$\mathcal{D}_{\text{KL}}(p||q) = \sum_i p_i \log \frac{p_i}{q_i} = \mathbb{E}_p \log \frac{p}{q}$$



KL divergence

KL divergence is a measure of how a probability distribution differs from another probability distribution

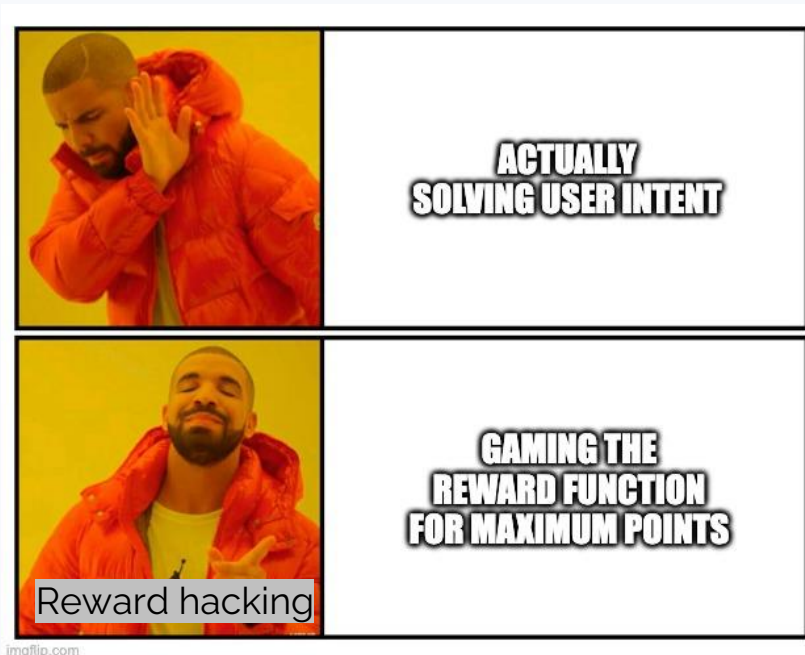
$$\mathcal{D}_{\text{KL}}(p||q) = \sum_i p_i \log \frac{p_i}{q_i} = \mathbb{E}_p \log \frac{p}{q}$$



RLHF – reinforcement learning from human feedback

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r(x, y)] - \beta \mathcal{D}_{\text{KL}} [\pi_{\theta}(y|x) || \pi_{\text{SFT}}(y|x)]$$

What if there is no KL divergence term?



RLHF problems

- Online method
- Requires 3 copies of LLM and 1 copy of Reward model
- Hard to stabilize

Reinforcement Learning with Online Interactions



Offline Reinforcement Learning

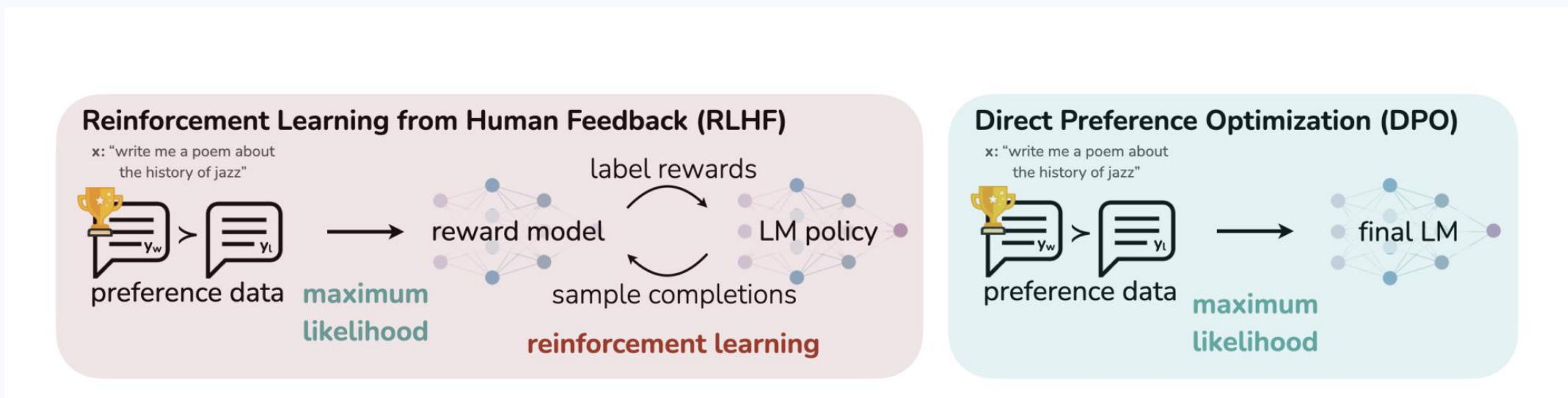


RLHF problems

- Online method
- Requires 3 copies of LLM and 1 copy of Reward model
- Hard to stabilize

So, we need offline method with only 1 LLM: **DPO**

DPO



$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\lambda \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \lambda \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

$$\text{Loss} = -\log \left(\sigma \left(r(y_w | x) - r(y_l | x) \right) \right) \quad - \text{Reward model loss}$$

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r(x, y)] - \beta \mathcal{D}_{\text{KL}} [\pi_{\theta}(y|x) || \pi_{\text{SFT}}(y|x)] \quad - \text{RLHF}$$

DPO in formulas

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[r(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x)} \right] = -\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right]$$

Lets denote: $Z(x) = \sum_y \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$, and: $\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$.

Rewrite $\mathcal{L}_{\text{RLHF}}$:

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[\log Z(x) - \log \frac{\pi_{\theta}(y|x)}{\frac{1}{Z(x)} \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right].$$

Since $Z(x)$ doesn't depend on θ , it has no effect on optimization.

$$\max_{\theta} \mathcal{L}_{\text{RLHF}} = \min_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[\frac{\pi_{\theta}(y|x)}{\frac{1}{Z(x)} \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right]$$

Therefore, the optimal policy π_{θ} becomes:

$$\pi_{\theta}(y|x) = \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right).$$

DPO in formulas

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[r(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x)} \right] = -\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right]$$

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{SFT}}(y|x)} + \beta \log Z(x)$$

Bradley-Terry model, so we'll cling to it:

$$p(y_a \succ y_r | x) = \sigma(r(x, y_a) - r(x, y_r))$$

The loss that we need is:

$$\mathcal{L}_{\text{DPO}} = \mathbb{E}_{(x, y_a, y_r) \sim \mathcal{D}} [p_{\theta}(y_a \succ y_r | x)]$$

where

$$\begin{aligned} p_{\theta}(y_a \succ y_r | x) &= \sigma \left(\left[\beta \log \frac{\pi_{\theta}(y_a|x)}{\pi_{\text{SFT}}(y_a|x)} + \beta \log Z(x) \right] - \left[\beta \log \frac{\pi_{\theta}(y_r|x)}{\pi_{\text{SFT}}(y_r|x)} + \beta \log Z(x) \right] \right) \\ &= \sigma \left(\beta \log \frac{\pi_{\theta}(y_a|x)}{\pi_{\text{SFT}}(y_a|x)} - \beta \log \frac{\pi_{\theta}(y_r|x)}{\pi_{\text{SFT}}(y_r|x)} \right) \end{aligned}$$

DPO in formulas

$$\begin{aligned} \nabla \mathcal{L}_{\text{DPO}} = \\ = -\beta \mathbb{E} \left[\underbrace{\sigma(\hat{r}_\theta(x, y_a) - \hat{r}_\theta(x, y_r))}_{\text{higher weight when reward estimate is wrong}} \left(\underbrace{\nabla_\theta \log \pi_\theta(y_a|x)}_{\uparrow \text{likelihood of } y_a} - \underbrace{\nabla_\theta \log \pi_\theta(y_r|x)}_{\downarrow \text{likelihood of } y_r} \right) \right], \end{aligned}$$

where $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{SFT}}(y|x)}$ is also a surrogate reward function of sorts.

Data

- Pre-training – large amounts of data, full internet
- SFT – prompts for assistant format, answers from assessors/models
- RLHF, DPO: prompts for assistant format from internet, pairwise answers generated by model from SFT, pairwise estimations from assessors/models

*SFT, RLHF parts can be repeated few times, 6 times for Llama 3 with synthetic data

Pre-training	15T tokens
SFT + DPO	10 million human-annotated assets

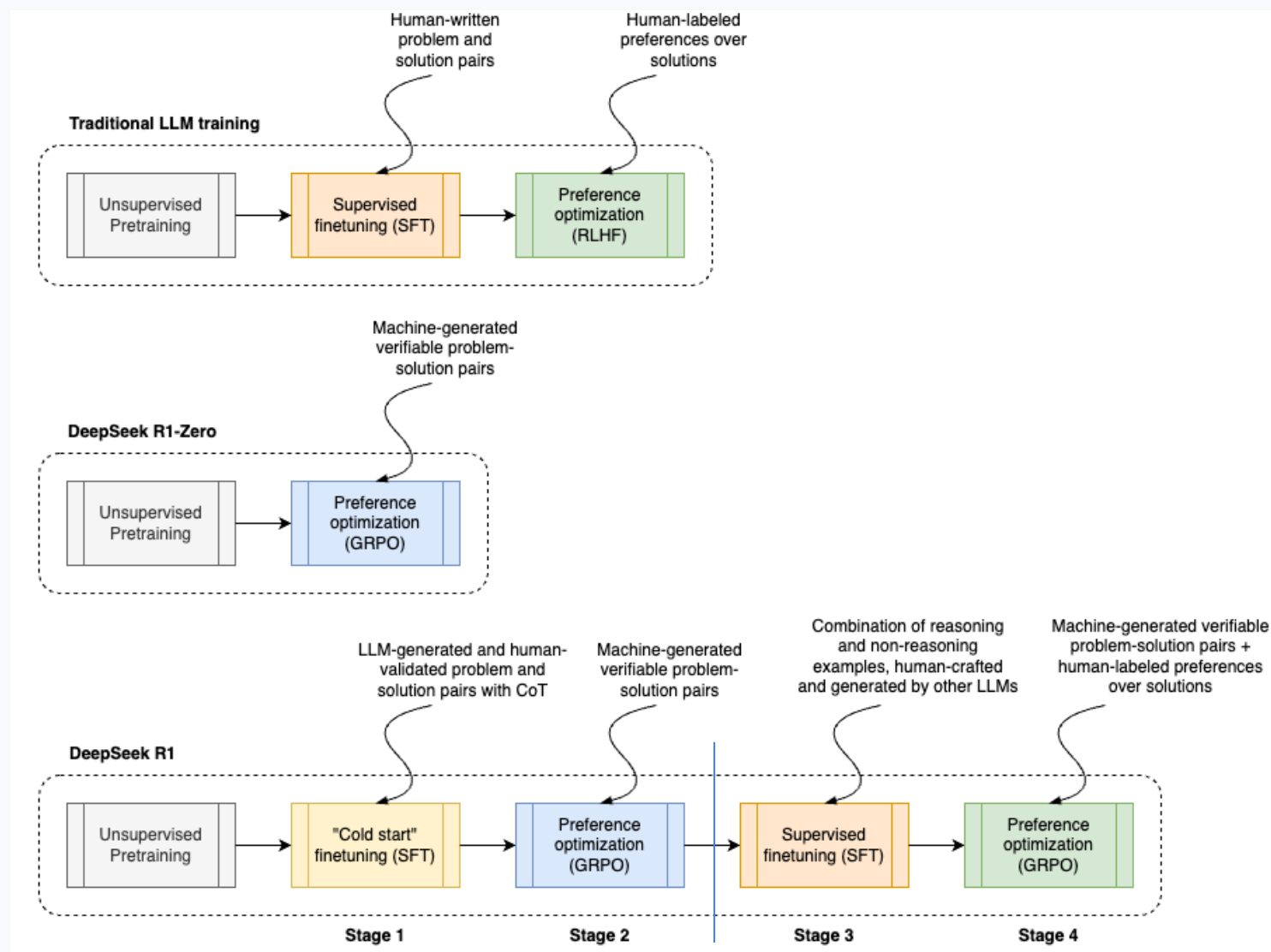
Chinchilla scaling laws

We want: training **Compute-Optimal** Large Language Models

$$L(N, D) = \underbrace{\frac{406.4}{N^{0.34}}}_{\text{finite model}} + \underbrace{\frac{410.7}{D^{0.28}}}_{\text{finite data}} + \underbrace{1.69}_{\text{irreducible}} \quad \text{with FLOPS} = 6ND.$$

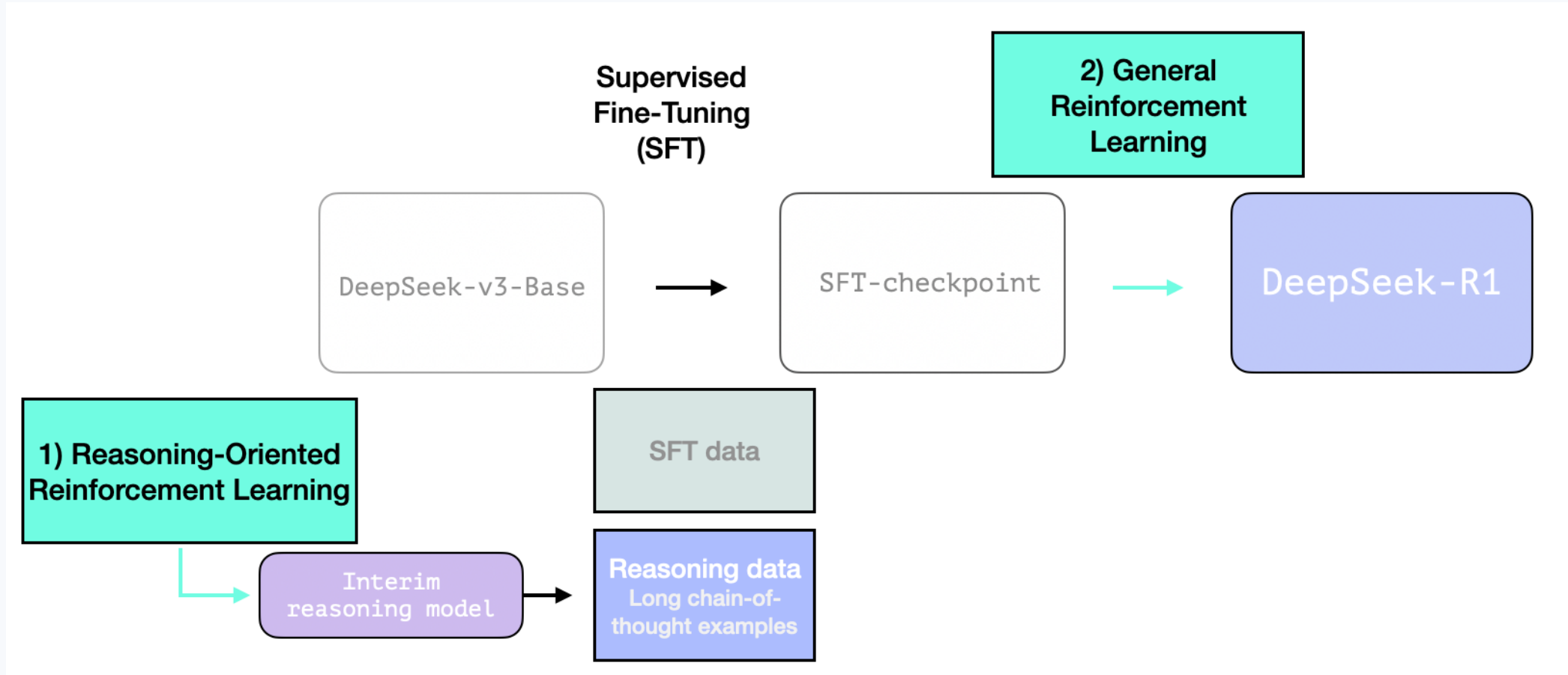
Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Bonus: Reasoning capabilities – R1



[link1](#), [link2](#), [link3](#)

Bonus: Reasoning capabilities – R1



[link1](#), [link2](#)

Thank you

